Universidad del Valle de Guatemala

Facultad de Ciencias y Humanidades

Programación Orientada a Objetos – Sección 21

Docente – Kevin Antonio Velásquez Aguilar



Ejercicio 6

Cristhopper Alejandro López Paz - 26544

Ingeniería en Ciencia de la Computación y Tecnologías de la Información

8/11/2026

1. Análisis del problema

El programa debe manejar integraciones de pagos, envíos y facturación. Cada proveedor puede realizar distintas operaciones, por lo que necesitamos una forma común de trabajar con ellos y agregar otros proveedores sin cambiar toda la lógica del programa.

Al iniciar se cargan diez integraciones. El menú permite listarlas con su estado de conexión, buscar una por su identificador para consultar su configuración y ordenarlas por latencia.

También se pueden simular cobros, reintegros y documentos. Cuando un proveedor no admite una operación, el programa debe omitirla y continuar funcionando.

2. Diseño de las clases

La clase padre es Integracion. Es abstracta y contiene los datos comunes: identificador, nombre, conexión, latencia, configuración e historial.

De ella heredan tres clases:

- IntegracionPago: realiza los cobros y reintegros simulados.
- IntegracionEnvio: permite simular el rastreo de un paquete.
- IntegracionFacturacion: genera un documento XML simulado.

CatalogoIntegraciones guarda los objetos y realiza la carga inicial, la búsqueda y el ordenamiento. También solicita las operaciones de los proveedores mediante interfaces.

Para aplicar MVC, el modelo está formado por el catálogo y las integraciones. VistaConsola muestra la información y lee los datos. Controlador maneja las opciones del menú y comunica la vista con el modelo.

Main solamente crea estos componentes e inicia el controlador.

3. Interfaces y polimorfismo

Interfaces:

- Transaccional: define cobrar y reintegrar.
- Documentable: define generarDocumento.
- Auditable: define el registro de eventos.
- Comparable<Integracion>: permite comparar las integraciones por latencia.

La clase padre implementa estos contratos. Las clases hijas sobrescriben las operaciones que pueden realizar.

Los métodos de Transaccional tienen una implementación por defecto que devuelve false. El método de Documentable devuelve una cadena vacía por defecto. Así, una integración de envíos puede recibir una solicitud de reintegro sin realizarla ni lanzar una excepción.

El catálogo aplica polimorfismo mediante parámetros de interfaz. Puede recibir distintos objetos y solicitar una operación usando el mismo contrato.

También tiene dos métodos llamados procesar: uno recibe un Transaccional y un monto y el otro recibe un Documentable y un texto. Esto es sobrecarga porque tienen el mismo nombre y diferentes parámetros.

4. Lista, ordenamiento y validaciones

Todos los objetos se guardan en una sola ArrayList<Integracion>. En ella se mezclan las integraciones de pagos, envíos y facturación.

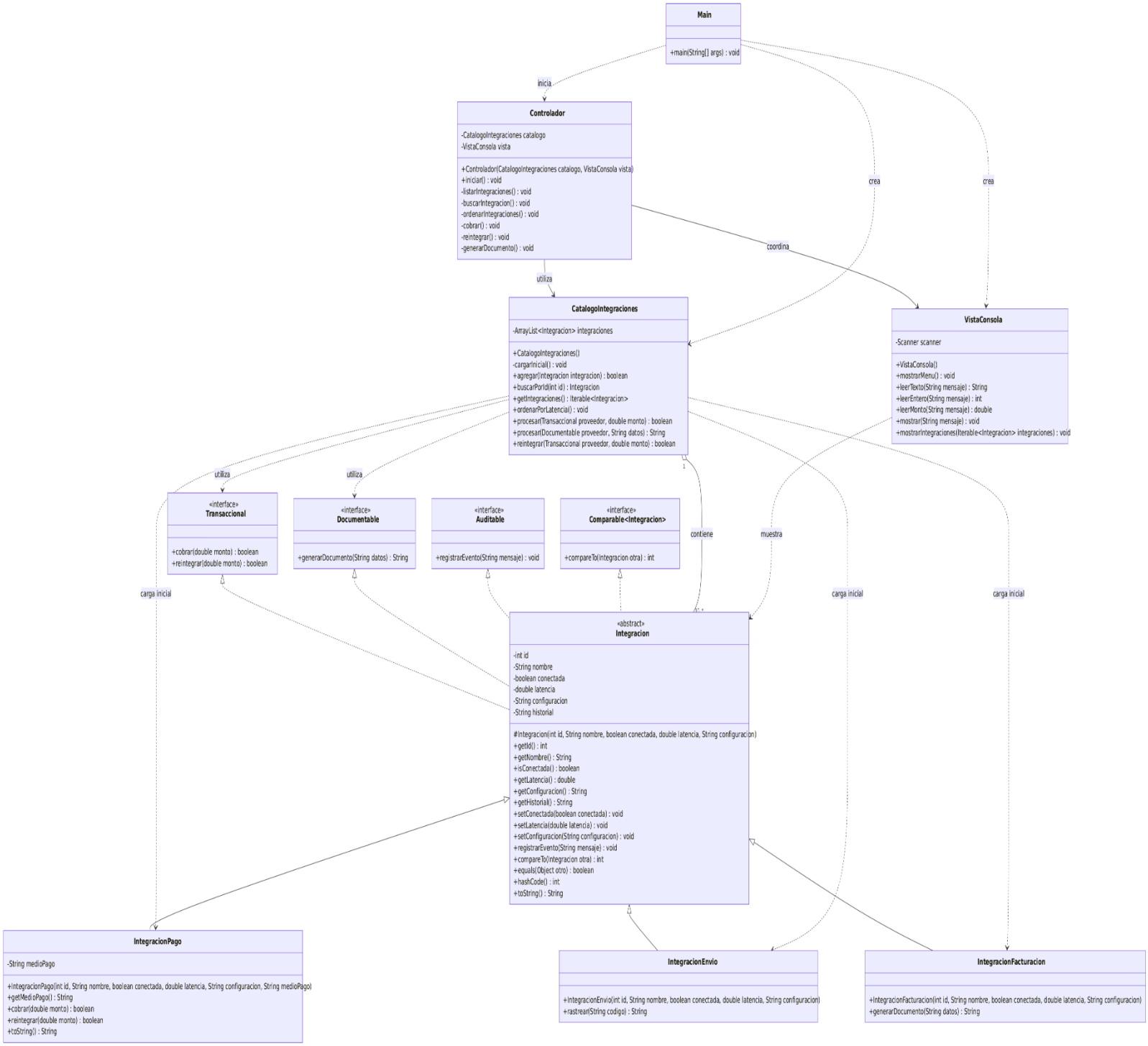
El método compareTo compara sus latencias. Después, Collections.sort ordena esa misma lista de menor a mayor, sin crear otro catálogo.

Los atributos son privados y se consultan mediante getters. Sólo hay setters para los datos que pueden cambiar: conexión, latencia y configuración.

Se comprueba que los identificadores sean positivos y no se repitan. Los textos obligatorios no pueden estar vacíos, la latencia debe ser finita y no negativa, y los montos deben ser finitos y mayores que cero.

toString permite mostrar los datos del objeto. equals compara la clase y el identificador, mientras que hashCode utiliza el identificador. El historial se actualiza mediante registrarEvento.

Diseño UML


Main
+main(String[] args) : void
inicia
crea
crea
Controlador
-CatalogoIntegraciones catalogo
-VistaConsola vista
+Controlador(CatalogoIntegraciones catalogo, VistaConsola vista)
+iniciar() : void
-listarIntegraciones() : void
-buscarIntegracion() : void
-ordenarIntegraciones() : void
-cobrar() : void
-reintegrar() : void
-generarDocumento() : void
coordina
utiliza
CatalogoIntegraciones
-ArrayList<Integracion> integraciones
+CatalogoIntegraciones()
-cargarInicial() : void
+agregar(Integracion integracion) : boolean
+buscarPorId(int id) : Integracion
+getIntegraciones() : Iterable<Integracion>
+ordenarPorLatencia() : void
+procesar(Transaccional proveedor, double monto) : boolean
+procesar(Documentable proveedor, String datos) : String
+reintegrar(Transaccional proveedor, double monto) : boolean
VistaConsola
-Scanner scanner
+VistaConsola()
+mostrarMenu() : void
+leerTexto(String mensaje) : String
+leerEntero(String mensaje) : int
+leerMonto(String mensaje) : double
+mostrar(String mensaje) : void
+mostrarIntegraciones(Iterable<Integracion> integraciones) : void
utiliza
utiliza
1
contiene
muestra
«interface»
Transaccional
+cobrar(double monto) : boolean
+reintegrar(double monto) : boolean
«interface»
Documentable
+generarDocumento(String datos) : String
«interface»
Auditable
+registrarEvento(String mensaje) : void
«interface»
Comparable<Integracion>
+compareTo(Integracion otra) : int
carga inicial
carga inicial
carga inicial
«abstract»
Integracion
-int id
-String nombre
-boolean conectada
-double latencia
-String configuracion
-String historial
#Integracion(int id, String nombre, boolean conectada, double latencia, String configuracion)
+getId() : int
+getNombre() : String
+isConectada() : boolean
+getLatencia() : double
+getConfiguracion() : String
+getHistorial() : String
+setConectada(boolean conectada) : void
+setLatencia(double latencia) : void
+setConfiguracion(String configuracion) : void
+registrarEvento(String mensaje) : void
+compareTo(Integracion otra) : int
+equals(Object otro) : boolean
+hashCode() : int
+toString() : String
IntegracionPago
-String medioPago
+IntegracionPago(int id, String nombre, boolean conectada, double latencia, String configuracion, String medioPago)
+getMedioPago() : String
+cobrar(double monto) : boolean
+reintegrar(double monto) : boolean
+toString() : String
IntegracionEnvio
+IntegracionEnvio(int id, String nombre, boolean conectada, double latencia, String configuracion)
+rastrear(String codigo) : String
IntegracionFacturacion
+IntegracionFacturacion(int id, String nombre, boolean conectada, double latencia, String configuracion)
+generarDocumento(String datos) : String